JPR Assignment 2

1. Explain inheritance. Define multi-level inheritance with syntax and example.

Ans. In Java, inheritance is a fundamental concept of Object-Oriented Programming (OOP) that allows one class to acquire the properties (fields) and behaviors (methods) of another.

Think of it as a way to create a "is-a" relationship between classes. For example, a Car "is-a" Vehicle.

Multi-level Inheritance

Multi-level inheritance occurs when a class extends another class, which in turn extends another class. This creates a "chain" of inheritance.

In this scenario, a child class acts as a parent for another class. For example, Class C inherits from Class B, and Class B inherits from Class A.

Syntax

In Java, the extends keyword is used to perform inheritance.

```
class Grandparent {
   // fields and methods
}
class Parent extends Grandparent {
   // fields and methods
}
class Child extends Parent {
   // fields and methods
}
```

Practical Example: Electronics Hierarchy

Let's look at a real-world example involving electronic devices.

```
// Grandparent Class
class Device {
   void powerOn() {
      System.out.println("Device is turning on...");
   }
}
// Parent Class (Inherits from Device)
class Smartphone extends Device {
   void connectToWifi() {
      System.out.println("Connecting to Wi-Fi...");
   }
}
// Child Class (Inherits from Smartphone)
class iPhone extends Smartphone {
   void useSiri() {
      System.out.println("Siri is listening...");
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      // Create an object of the bottom-level class
      iPhone myPhone = new iPhone();
      // Accessing methods from all levels of the chain
      myPhone.powerOn();       // From Device
      myPhone.connectToWifi(); // From Smartphone
      myPhone.useSiri();       // From iPhone
   }
}
```

2. Define interface. How to implement interface, accessing interface variable and extending interface.

Ans. In Java, an Interface is a blueprint of a class. It is a collection of abstract methods (methods without a body) and static constants. While a class describes *what an object is*, an interface describes *what an object can do*.

1. Interface

An interface tells a class what it must do, but not how to do it. It provides a way to achieve 100% abstraction and multiple inheritance in Java.

- All methods in an interface are implicitly public and abstract (until Java 8).
- All variables are implicitly public, static, and final.

2. Implement an Interface

To use an interface, a class must "implement" it using the implements keyword. The class is then obligated to provide concrete implementations for all methods declared in the interface.

```
interface Animal {
   void makeSound(); // Abstract method
}
class Cat implements Animal {
   public void makeSound() {
      System.out.println("Meow!");
   }
}
```

3. Accessing Interface Variables

Because interface variables are static and final, they are essentially constants. You can access them directly using the Interface Name, even without creating an instance of a class.

```
interface Config {
   int TIMEOUT = 30; // public static final by default
}
public class Main {
   public static void main(String[] args) {
      // Accessed via the interface name
```

```
        System.out.println("Timeout is: " + Config.TIMEOUT);
    }
}
```

4. Extending an Interface

Just as a class can extend another class, an interface can extend another interface using the extends keyword. This allows you to build upon existing requirements.

A class implementing the "child" interface must implement all methods from both the parent and the child interfaces.

```
interface Drawable {
    void draw();
}
// Interface inheritance
interface Colorable extends Drawable {
    void applyColor();
}
class Square implements Colorable {
    public void draw() {
        System.out.println("Drawing square...");
    }
    public void applyColor() {
        System.out.println("Applying red color...");
    }
}
```

3. Explain what do you mean by packages with its type.

Ans. In Java, a Package is a mechanism used to group related classes, interfaces, and sub-packages together. Think of it like a folder on your computer that keeps your files organized and prevents naming conflicts.

Types of Packages

Java packages are categorized into two main types:

A. Built-in Packages

These are pre-defined packages provided by the Java Development Kit (JDK). They contain classes for common tasks like input/output, networking, and utility functions.

Package	Description
java.lang	Contains fundamental classes (e.g., String, Math, System). It is imported automatically.
java.util	Contains utility classes like Scanner, ArrayList, and Date.
java.io	Contains classes for input and output streams.
java.net	Contains classes for networking operations.

Package	Description
java.awt	Used for creating Graphical User Interfaces (GUI).

B. User-defined Packages

These are packages created by developers to organize their own project code.

How to create and use one:

1. **Declare:** Use the package keyword at the very top of your source file.
2. **Import:** Use the import keyword to use classes from a different package.

```
// File: MyClass.java
package com.myproject.utils; // User-defined package
public class MyClass {
   public void display() {
      System.out.println("Hello from a custom package!");
   }
}
```

4. Define import statement and static import with syntax.

Ans. In Java, both import and static import are used to improve code readability by allowing you to use short names instead of "Fully Qualified Names" (the complete package path).

1. The import Statement

The standard import statement is used to bring classes and interfaces from other packages into the current file's visibility. Without it, you would have to type the full path every time you use a class (e.g., java.util.Scanner).

Syntax

There are two ways to use the standard import:

1. **Specific Import:** Imports only a single class.
    - import package.subpackage.ClassName;
2. **Wildcard Import:** Imports all classes within a specific package.
    - import package.subpackage.*;

Example

```
import java.util.ArrayList; // Specific import
public class Test {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>(); // No need for java.util.ArrayList
   }
}
```

2. The static import Statement

Introduced in Java 5, the static import allows you to access static members (fields and methods) of a class directly without qualifying them with the class name.

Syntax

1. **Specific Static Import:** Imports one specific static member.
    - import static package.ClassName.staticMemberName;

2. **Static Wildcard Import:** Imports all static members of a class.
    - import static package.ClassName.*;

Example

Normally, you write Math.sqrt(). With a static import, you can just write sqrt().

```
import static java.lang.Math.PI;   // Static field
import static java.lang.Math.sqrt; // Static method
public class Circle {
   void calculate(double radius) {
      double area = PI * radius * radius; // No Math.PI needed
      System.out.println(sqrt(area));     // No Math.sqrt needed
   }
}
```

5. Explain method overriding with a program.

Ans. Method Overriding occurs when a subclass (child class) provides a specific implementation for a method that is already defined in its superclass (parent class).

It is the primary way Java achieves Runtime Polymorphism, as the JVM determines which method to execute at runtime based on the actual object type, not the reference type.

1. Basic Example Program

In this example, the Dog class overrides the makeSound method of the Animal class to provide its own specific behavior.

```
class Animal {
   // Overridden method
   void makeSound() {
      System.out.println("The animal makes a generic sound.");
   }
}
class Dog extends Animal {
   // Overriding method
   @Override
   void makeSound() {
      System.out.println("The dog barks: Woof! Woof!");
   }
}
public class Main {
   public static void main(String[] args) {
      Animal myAnimal = new Animal();
      Animal myDog = new Dog(); // Parent reference, Child object (Upcasting)
      myAnimal.makeSound(); // Prints: The animal makes a generic sound.
      myDog.makeSound();    // Prints: The dog barks: Woof! Woof!
   }
}
```